12_Multiple_Regression_Yuliia_Chernysheva

November 11, 2026

Yuliia Chernysheva
November 11, 2026
CSC 2267: Multiple Regression Assignment

Dataset Summary

This dataset contains information about automobile miles per gallon (`MPG`) based on the number of cylinders, the displacement, the horsepower, the weight, the acceleration, the model year, the origin, and the car.

Import libraries

[1]:
```
import pandas as pd
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import mutual_info_regression
```

1 Task 1: Read the data file from the Seaborn website

[2]:
```
# Load the automobile MPG dataset directly from the Seaborn repository
data = sns.load_dataset('mpg')
```

2 Task 2: Display the first five rows of data.

[3]:
```
# Preview the first five records of the raw dataset
display(data.head())
```

```
    mpg  cylinders  displacement  horsepower  weight  acceleration  \
0  18.0          8         307.0       130.0    3504          12.0
1  15.0          8         350.0       165.0    3693          11.5
2  18.0          8         318.0       150.0    3436          11.0
3  16.0          8         304.0       150.0    3433          12.0
4  17.0          8         302.0       140.0    3449          10.5

   model_year origin                       name
```

```
0           70    usa  chevrolet chevelle malibu
1           70    usa           buick skylark 320
2           70    usa          plymouth satellite
3           70    usa               amc rebel sst
4           70    usa                 ford torino
```

3 Task 3: Use the info() method to display the data types for the columns

[4]:
```
# Inspect structural summary, column data types, and check for null values
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    392 non-null    float64
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model_year    398 non-null    int64
 7   origin        398 non-null    object
 8   name          398 non-null    object
dtypes: float64(4), int64(3), object(2)
memory usage: 28.1+ KB
```

4 Task 4: Get the brand from the name column and store it in a new column named brand

[5]:
```
# Extract the first word from the car name string as the brand
data["brand"] = data.apply(lambda x: x['name'].split()[0], axis=1)

# Verify the newly created "brand" column
data.head()
```

[5]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	\
0	18.0	8	307.0	130.0	3504	12.0	
1	15.0	8	350.0	165.0	3693	11.5	
2	18.0	8	318.0	150.0	3436	11.0	
3	16.0	8	304.0	150.0	3433	12.0	
4	17.0	8	302.0	140.0	3449	10.5	

	model_year	origin	name	brand

```
0          70    usa  chevrolet chevelle malibu  chevrolet
1          70    usa          buick skylark 320      buick
2          70    usa         plymouth satellite   plymouth
3          70    usa              amc rebel sst        amc
4          70    usa                ford torino       ford
```

5 Task 5: Drop the name column, and then display the first five rows of data.

[6]:
```
# Drop the original 'name' column since brand is now isolated
cars_data = data.drop(columns=['name'])

# Display the first five rows of the updated DataFrame
display(cars_data.head())
```

```
    mpg  cylinders  displacement  horsepower  weight  acceleration  \
0  18.0          8         307.0       130.0    3504          12.0
1  15.0          8         350.0       165.0    3693          11.5
2  18.0          8         318.0       150.0    3436          11.0
3  16.0          8         304.0       150.0    3433          12.0
4  17.0          8         302.0       140.0    3449          10.5

   model_year origin      brand
0          70    usa  chevrolet
1          70    usa      buick
2          70    usa   plymouth
3          70    usa        amc
4          70    usa       ford
```

6 Task 6: Display all of the unique values in the brand column

[7]:
```
# List all distinct brand names to inspect for typos and duplicates
cars_data.brand.unique()
```

[7]:
```
array(['chevrolet', 'buick', 'plymouth', 'amc', 'ford', 'pontiac',
       'dodge', 'toyota', 'datsun', 'volkswagen', 'peugeot', 'audi',
       'saab', 'bmw', 'chevy', 'hi', 'mercury', 'opel', 'fiat',
       'oldsmobile', 'chrysler', 'mazda', 'volvo', 'renault', 'toyouta',
       'maxda', 'honda', 'subaru', 'chevroelt', 'capri', 'vw',
       'mercedes-benz', 'cadillac', 'mercedes', 'vokswagen', 'triumph',
       'nissan'], dtype=object)
```

7 Task 7: Review the brand data, and then fix the spelling errors in the brand column.

[8]:
```
# Define dictionary mapping misspelled or alternate brand names to standard␣
 ↪forms
corrections = {
    'chevy': 'chevrolet',
    'chevroelt': 'chevrolet',
    'toyouta': 'toyota',
    'maxda': 'mazda',
    'vw': 'volkswagen',
    'vokswagen': 'volkswagen',
    'mercedes-benz': 'mercedes'
}

# Standardize brand names across the dataset
cars_data["brand"] = cars_data["brand"].replace(corrections)

# List all distinct brand names
cars_data.brand.unique()
```

[8]:
```
array(['chevrolet', 'buick', 'plymouth', 'amc', 'ford', 'pontiac',
       'dodge', 'toyota', 'datsun', 'volkswagen', 'peugeot', 'audi',
       'saab', 'bmw', 'hi', 'mercury', 'opel', 'fiat', 'oldsmobile',
       'chrysler', 'mazda', 'volvo', 'renault', 'honda', 'subaru',
       'capri', 'mercedes', 'cadillac', 'triumph', 'nissan'], dtype=object)
```

8 Task 8: Drop all rows from the DataFrame that have missing values

[9]:
```
# Remove any records with missing values
cars_data = cars_data.dropna()
```

9 Task 9: Get the two categorical columns (the one with the object data type), and store them in a list. Then, convert those columns to dummy variables, and store the results in a new DataFrame.

[10]:
```
# Specify the two categorical columns
cat_columns = ['origin','brand']

# Generate dummy variables
dummies = pd.get_dummies(cars_data[cat_columns])
```

```
# Inspect the structure and column count of the dummy DataFrame
dummies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 33 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   origin_europe     392 non-null    bool
 1   origin_japan      392 non-null    bool
 2   origin_usa        392 non-null    bool
 3   brand_amc         392 non-null    bool
 4   brand_audi        392 non-null    bool
 5   brand_bmw         392 non-null    bool
 6   brand_buick       392 non-null    bool
 7   brand_cadillac    392 non-null    bool
 8   brand_capri       392 non-null    bool
 9   brand_chevrolet   392 non-null    bool
 10  brand_chrysler    392 non-null    bool
 11  brand_datsun      392 non-null    bool
 12  brand_dodge       392 non-null    bool
 13  brand_fiat        392 non-null    bool
 14  brand_ford        392 non-null    bool
 15  brand_hi          392 non-null    bool
 16  brand_honda       392 non-null    bool
 17  brand_mazda       392 non-null    bool
 18  brand_mercedes    392 non-null    bool
 19  brand_mercury     392 non-null    bool
 20  brand_nissan      392 non-null    bool
 21  brand_oldsmobile  392 non-null    bool
 22  brand_opel        392 non-null    bool
 23  brand_peugeot     392 non-null    bool
 24  brand_plymouth    392 non-null    bool
 25  brand_pontiac     392 non-null    bool
 26  brand_renault     392 non-null    bool
 27  brand_saab        392 non-null    bool
 28  brand_subaru      392 non-null    bool
 29  brand_toyota      392 non-null    bool
 30  brand_triumph     392 non-null    bool
 31  brand_volkswagen  392 non-null    bool
 32  brand_volvo       392 non-null    bool
dtypes: bool(33)
memory usage: 15.7 KB
```

10 Task 10: Drop categorical columns and join dummy variables into dataDummies

[11]:
```
# Drop the original categorical columns from the dataset
dataDummies = cars_data.drop(columns=cat_columns)

# Join the generated dummy variables to the numeric DataFrame
dataDummies = dataDummies.join(dummies)

# Display the resulting columns and data types
dataDummies.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 392 entries, 0 to 397
Data columns (total 40 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   mpg               392 non-null    float64
 1   cylinders         392 non-null    int64
 2   displacement      392 non-null    float64
 3   horsepower        392 non-null    float64
 4   weight            392 non-null    int64
 5   acceleration      392 non-null    float64
 6   model_year        392 non-null    int64
 7   origin_europe     392 non-null    bool
 8   origin_japan      392 non-null    bool
 9   origin_usa        392 non-null    bool
 10  brand_amc         392 non-null    bool
 11  brand_audi        392 non-null    bool
 12  brand_bmw         392 non-null    bool
 13  brand_buick       392 non-null    bool
 14  brand_cadillac    392 non-null    bool
 15  brand_capri       392 non-null    bool
 16  brand_chevrolet   392 non-null    bool
 17  brand_chrysler    392 non-null    bool
 18  brand_datsun      392 non-null    bool
 19  brand_dodge       392 non-null    bool
 20  brand_fiat        392 non-null    bool
 21  brand_ford        392 non-null    bool
 22  brand_hi          392 non-null    bool
 23  brand_honda       392 non-null    bool
 24  brand_mazda       392 non-null    bool
 25  brand_mercedes    392 non-null    bool
 26  brand_mercury     392 non-null    bool
 27  brand_nissan      392 non-null    bool
 28  brand_oldsmobile  392 non-null    bool
 29  brand_opel        392 non-null    bool
```

```
 30  brand_peugeot     392 non-null    bool
 31  brand_plymouth    392 non-null    bool
 32  brand_pontiac     392 non-null    bool
 33  brand_renault     392 non-null    bool
 34  brand_saab        392 non-null    bool
 35  brand_subaru      392 non-null    bool
 36  brand_toyota      392 non-null    bool
 37  brand_triumph     392 non-null    bool
 38  brand_volkswagen  392 non-null    bool
 39  brand_volvo       392 non-null    bool
dtypes: bool(33), float64(4), int64(3)
memory usage: 37.1 KB
```

11 Task 11: Rescale the data in the numeric columns, and then display the rescaled data

[12]:
```
# Initialize StandardScaler to standardize numeric features
scaler = StandardScaler()

# List numeric predictor columns (excluding the target 'mpg' and dummy variables
num_cols = ['cylinders',␣
 ↪'displacement','horsepower','weight','acceleration','model_year']

# Fit scaler and transform numeric features in-place
dataDummies[num_cols] = scaler.fit_transform(dataDummies[num_cols])

# Display the standardized dataset
dataDummies.head()
```

[12]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	\
0	18.0	1.483947	1.077290	0.664133	0.620540	-1.285258	
1	15.0	1.483947	1.488732	1.574594	0.843334	-1.466724	
2	18.0	1.483947	1.182542	1.184397	0.540382	-1.648189	
3	16.0	1.483947	1.048584	1.184397	0.536845	-1.285258	
4	17.0	1.483947	1.029447	0.924265	0.555706	-1.829655	

	model_year	origin_europe	origin_japan	origin_usa	…	brand_peugeot	\
0	-1.625315	False	False	True	…	False	
1	-1.625315	False	False	True	…	False	
2	-1.625315	False	False	True	…	False	
3	-1.625315	False	False	True	…	False	
4	-1.625315	False	False	True	…	False	

	brand_plymouth	brand_pontiac	brand_renault	brand_saab	brand_subaru	\
0	False	False	False	False	False	
1	False	False	False	False	False	

```
2            True          False          False       False         False
3           False          False          False       False         False
4           False          False          False       False         False

   brand_toyota  brand_triumph  brand_volkswagen  brand_volvo
0         False          False             False        False
1         False          False             False        False
2         False          False             False        False
3         False          False             False        False
4         False          False             False        False

[5 rows x 40 columns]
```

12 Task 12: Display the correlation data for the mpg column

[13]:
```
# Calculate Pearson correlation matrix, filter for 'mpg', and sort in␣
↪descending order
dataDummies.corr()[['mpg']].sort_values(by='mpg',ascending=False)
```

[13]:
```
                     mpg
mpg             1.000000
model_year      0.580541
origin_japan    0.451454
acceleration    0.423329
brand_volkswagen  0.262610
brand_datsun    0.245564
brand_honda     0.245092
origin_europe   0.244313
brand_toyota    0.161369
brand_mazda     0.150744
brand_fiat      0.101222
brand_subaru    0.092209
brand_nissan    0.081448
brand_triumph   0.074960
brand_renault   0.070083
brand_audi      0.056537
brand_opel      0.030012
brand_capri     0.010082
brand_saab      0.005915
brand_mercedes  0.005867
brand_peugeot   0.004473
brand_bmw       0.002794
brand_cadillac -0.033954
brand_volvo    -0.037255
brand_oldsmobile -0.048693
brand_dodge    -0.049286
```

```
brand_plymouth   -0.065513
brand_pontiac    -0.090860
brand_hi         -0.093721
brand_mercury    -0.094336
brand_chrysler   -0.098832
brand_buick      -0.116456
brand_chevrolet  -0.152788
brand_amc        -0.187560
brand_ford       -0.190289
origin_usa       -0.565161
cylinders        -0.777618
horsepower       -0.778427
displacement     -0.805127
weight           -0.832244
```

13 Task 13: Split the data into test and training datasets

[14]:
```
# Split the dataset into 80% training and 20% testing sets
# with fixed random seed
cars_train, cars_test = train_test_split(dataDummies,
                                         test_size=0.2,
                                         random_state=20)
```

14 Task 14: Use a for loop with the feature selection model to test different numbers of features

[15]:
```
model = LinearRegression()
test_scores = []
train_scores = []

# Loop across varying numbers of features (k from 1 up to total predictor count)
for i in range(1, len(cars_train.columns)):
    # Configure feature selector using mutual information criterion
    fs = SelectKBest(score_func=mutual_info_regression, k=i)

    # Fit selector on training predictors and target variable
    fs.fit(cars_train.drop(columns=['mpg']), cars_train['mpg'])

    # Transform both training and test features to the top k selected attributes
    x_train_fs = fs.transform(cars_train.drop(columns=['mpg']))
    x_test_fs = fs.transform(cars_test.drop(columns=['mpg']))

    # Train linear regression model on selected subset of features
    model.fit(x_train_fs, cars_train['mpg'])
```

```
    # Evaluate R-squared coefficient on test and train partitions
    test_score = model.score(x_test_fs, cars_test['mpg'])
    train_score = model.score(x_train_fs, cars_train['mpg'])

    # Store scores for visualization
    test_scores.append(test_score)
    train_scores.append(train_score)
```

15 Task 15: Plot the test and training scores with a line plot based on the number of features

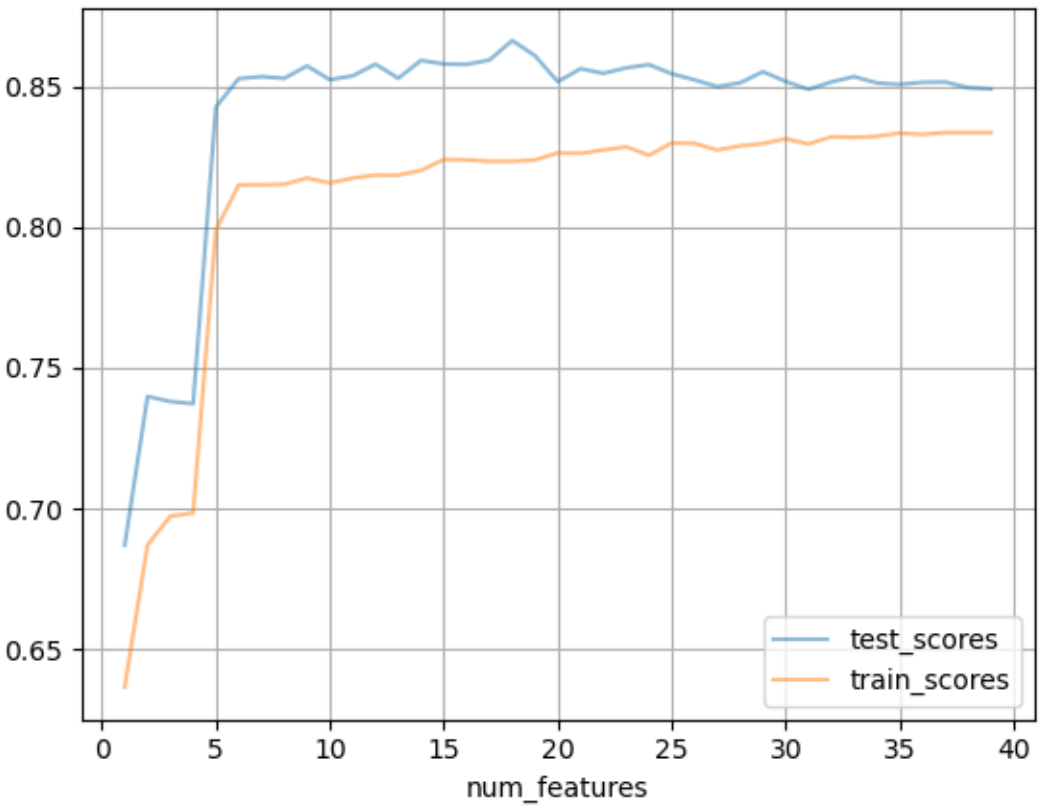
[16]:
```
# Create a DataFrame to hold training and testing R2 scores
df = pd.DataFrame(data={'test_scores':test_scores, 'train_scores':train_scores})

# reset index to create a feature count axis
df.reset_index(inplace=True)
df.rename(columns={'index':'num_features'}, inplace=True)

# Adjust zero-based index to 1-based feature count
df.num_features = df.num_features + 1

# Plot performance curves comparing train and test R2 scores across feature␣
↪counts
df.plot(x='num_features', y=['test_scores','train_scores'], grid =True, alpha=0.
↪5)
```

[16]:
```
<Axes: xlabel='num_features'>
```



16 Conclusion

Based on the feature selection plot:

- A sharp increase in performance occurs at **5 features**, where the testing R^2 score jumps from ~0.74 to over **0.84**, and the training score reaches **0.80**.
- Beyond 5 features, both scores plateau with only marginal variations, indicating diminishing returns and unnecessary model complexity.
- Therefore, a model with **5 features** provides the optimal balance of predictive accuracy, simplicity, and generalizability for predicting automobile MPG.

[ ]: